❷ The value of ViewData[“Title”] will be set to “Home Page”.

❸ You can set keys in the ViewData dictionary directly.

You can display the values in the template in the same way as before:

```
<h1>@ViewData["Title"]</h3>
<h2>@ViewData["Subtitle"]</h3>
```

TIP I don’t find the `[ViewData]` attribute especially useful, but it’s another feature to look out for. Instead, I create a set of global, static constants for any `ViewData` keys, and I reference those instead of typing `"Title"` repeatedly. You’ll get IntelliSense for the values, they’re refactor-safe, and you’ll avoid hard-to-spot typos.

As I mentioned previously, there are mechanisms besides `PageModel` properties and `ViewData` that you can use to pass data around, but these two are the only ones I use personally, as you can do everything you need with them. As a reminder, always use `PageModel` properties where possible, as you benefit from strong typing and IntelliSense. Only fall back to `ViewData` for values that need to be accessed *outside* of your Razor view.

You’ve had a small taste of the power available to you in Razor templates, but in the next section we’ll dive a little deeper into some of the available C# capabilities.

17.3 Creating dynamic web pages with Razor

You might be glad to know that pretty much anything you can do in C# is possible in Razor syntax. Under the covers, the .cshtml files are compiled into normal C# code (with `string` for the raw HTML sections), so whatever weird and wonderful behavior you need can be created!